

Module 4 Lab Session 2: Services Done Right

Field	Value
Module	M4 ASP.NET Core 10 Fundamentals
Exercises	2 (Captive Dependencies), 3 (Options Pattern), 4 (Structured Logging)
Assessment Tiers	Tier 2 DI (LO 4.3, 4.4) + Tier 3 Config (LO 4.5) + Tier 4 Logging (LO 4.6)

Prerequisites Check Before You Start

Your TmsApi project must contain:

- Program.cs with the canonical pipeline order from Session 1 UseRouting, UseAuthentication, UseAuthorization, all before any MapGet
- RequestLoggingMiddleware.cs registered as the **first** middleware in the pipeline
- A working GET /api/assessments/results endpoint that returns 401 Unauthorized for anonymous calls
- An X-Correlation-Id header on every response

If any of these are missing or incomplete, go back to Session 1. The three exercises in this session depend on a working pipeline and a Correlation ID. Without them you will spend time chasing symptoms instead of learning the lesson.

What This Session Is Really About

Session 2 covers three topics that look unrelated until you see them break in production:

1. **DI lifetimes done wrong** (Exercise 2) a singleton holds a scoped service, the database connection pool exhausts at 3 AM
2. **Configuration done wrong** (Exercise 3) a required setting is missing, the app starts fine, then the first user to trigger that code path crashes the request
3. **Logging done wrong** (Exercise 4) the logs work, but they are unsearchable strings, so when something goes wrong you cannot find the one log line that matters

These are the **silent-failure** topics of ASP.NET Core. None of them produce a compile error. None of them fail in development with one user. They fail at 2 AM under real load with a real customer waiting. Your job in Session 2 is to learn the discipline that prevents each of them DI lifetime correctness, validated configuration, and structured logging as one cohesive arc, because in real systems they fail together.

TMS Service Foundation

Before you continue, you need the enrollment service that the next exercises depend on. This is the same TMS domain from M1 now exposed as a service the web layer can inject.

Create a new file called EnrollmentService.cs:

```
// --- The contract ---
public interface IEnrollmentService
{
    Task<EnrollmentRecord> EnrollAsync(string studentId, string courseCode);
    Task<EnrollmentRecord?> GetByIdAsync(string id);
    Task<IReadOnlyList<EnrollmentRecord>> GetAllAsync();
    Task<bool> DeleteAsync(string id);
}

// --- The in-memory implementation ---
public class EnrollmentService : IEnrollmentService
{
    private readonly Dictionary<string, EnrollmentRecord> _store = new();
    private readonly ILogger<EnrollmentService> _logger;

    public EnrollmentService(ILogger<EnrollmentService> logger)
    {
        _logger = logger;
    }

    public Task<EnrollmentRecord> EnrollAsync(string studentId, string courseCode)
    {
        var id = Guid.NewGuid().ToString("N")[..8];
        var record = new EnrollmentRecord(id, studentId, courseCode, DateTime.UtcNow);
        _store[id] = record;

        _logger.LogInformation(
            "Enrolled {StudentId} in {CourseCode} record {EnrollmentId}",
            studentId, courseCode, id);

        return Task.FromResult(record);
    }

    public Task<EnrollmentRecord?> GetByIdAsync(string id)
    {
        _store.TryGetValue(id, out var record);
        return Task.FromResult(record);
    }

    public Task<IReadOnlyList<EnrollmentRecord>> GetAllAsync()
```

```

    {
        IReadOnlyList<EnrollmentRecord> all = _store.Values.ToList();
        return Task.FromResult(all);
    }

    public Task<bool> DeleteAsync(string id)
    {
        var removed = _store.Remove(id);
        return Task.FromResult(removed);
    }
}

// --- The data shape ---
public record EnrollmentRecord(
    string Id,
    string StudentId,
    string CourseCode,
    DateTime EnrolledAt);

```

Do **not** register it in Program.cs yet Exercise 2 will walk you through registration and lifetime decisions.

Troubleshooting

- EnrollmentRecord is a record type (immutable by default). If you need to review records, revisit M1 Exercise 3.
- The Task.FromResult calls simulate async I/O. In M5 these become real database calls with EF Core.

Exercise 2: The Memory Leak (Captive Dependencies)

Context: The TMS has a background worker that recalculates scholarships every hour. The server crashed with an OutOfMemoryException. The investigation traced the crash to a **Scoped** service (IEnrollmentService) being held by a **Singleton** (EnrollmentWorker) a **captive dependency**.

Quick reference the three lifetimes

- **Transient:** New instance every time it is resolved.
- **Scoped:** One instance per HTTP request; disposed when the request ends.
- **Singleton:** One instance for the entire application lifetime.

If a Scoped service (for example something that should track **this request's** enrollments) is captured inside a Singleton, it can live across requests. Under load, connections or in-memory state leak or go **stale** another request may see the wrong student's data.

Prerequisite (read once): Many HTTP requests can be **in flight at the same time**. Each request should get its **own** scoped services. A singleton lives forever it must **not** keep a reference to a scoped instance created for an old request.

First, make the failure visible

Step A Buggy registration (temporary): Implement `EnrollmentWorker` so its constructor takes `IEnrollmentService` **directly** (not `IServiceScopeFactory` yet). Register:

```
builder.Services.AddSingleton<EnrollmentWorker>();
builder.Services.AddScoped<IEnrollmentService, EnrollmentService>();
```

Add **host validation** so the container catches illegal lifetime wiring early:

```
builder.Host.UseDefaultServiceProvider(options =>
{
    options.ValidateScopes = true;
    options.ValidateOnBuild = true;
});
```

Run: `dotnet run`

Expected (the “good” failure): The app throws at startup or on first resolve with an error similar to:

Cannot consume scoped service 'IEnrollmentService' from singleton 'EnrollmentWorker'.

That message IS the captive dependency detector working. Read it carefully it names the two lifetimes that clash.

Step B Confirm the failure: If the app refused to start in Step A, that IS the expected failure. Read the error message carefully it names both lifetimes. Skip ahead to “Your Task” below.

If the app somehow started (unlikely with `ValidateOnBuild = true`), add this smoke-test route to `Program.cs` and run concurrent requests to expose the bug:

```
app.MapGet("/api/enrollments/worker-smoke", (EnrollmentWorker worker) =>
{
    worker.ProcessBatch();
    return Results.Ok("processed");
});
```

From **PowerShell** (replace the base URL with yours from `dotnet run`):

```
$base = "http://localhost:5000" # or https://localhost:7xxx match your terminal
1..15 | ForEach-Object -Parallel {
    Invoke-WebRequest -Uri "$using:base/api/enrollments/worker-smoke" -UseBasicParsing | Out-Null
} -ThrottleLimit 15
```

Expected: Exceptions or inconsistent data under parallel calls the scoped service is being shared across requests.

Troubleshooting: If nothing fails, confirm `ValidateScopes = true` is set. If the app refuses to start after the singleton registration, that IS the expected failure proceed to the fix below.

Your Task: Fix the Captive Dependency

Inject **IServiceScopeFactory** into the singleton and create a **short-lived scope** each time the worker runs. Resolve **IEnrollmentService** **from that scope** only.

// These registrations are given do NOT change them:

```
builder.Services.AddSingleton<EnrollmentWorker>();  
builder.Services.AddScoped<IEnrollmentService, EnrollmentService>();
```

// Inside EnrollmentWorker.cs...

```
public class EnrollmentWorker(IServiceScopeFactory scopeFactory)  
{  
    public void ProcessBatch()  
    {  
        // TODO 2: Create a short-lived scope using the injected factory.  
        // Stuck? using var scope = factory.CreateScope();  
  
        // TODO 3: Resolve the scoped service from the new scope's provider.  
        // Stuck? var svc = scope.ServiceProvider.GetRequiredService<IEnrollmentService>();  
  
        // TODO 4: Use the service, then let the 'using' block dispose the scope  
        // and its scoped services automatically.  
    }  
}
```

Run / Call / Expected / Common failure

- **Run:** dotnet run after implementing **IServiceScopeFactory**.
- **Call:** Same as Step A or B startup should succeed; if you have worker-smoke, run the PowerShell parallel block again.
- **Expected after fix:** No Cannot consume scoped service... at startup; under parallel calls, behavior is stable (no cross-request stale state for scoped work done **inside** `ProcessBatch`).
- **Common failure:** Forgetting using on the scope scoped services never dispose.
- **Common failure:** Resolving **IEnrollmentService** from the **root** `app.Services` inside the singleton still wrong. Always `CreateScope()` first.

Exercise 3: The Silent Crash (Options Pattern)

Context: The TMS relies on an external payment gateway for tuition processing (measured in Ethiopian Birr). The appsettings.json is missing the GatewayUrl key.

Because the previous developer used IConfiguration["GatewayUrl"] inline during checkout, the application runs perfectly until a student tries to pay, throwing a massive server error.

[!CAUTION] **Think about this before coding:** If an application is missing a critical configuration setting, is it better for it to crash entirely upon startup, or crash later when a user tries to use the specific feature?

Your Task: Validate Configuration at Startup

Build a strongly-typed options class and wire it so the app refuses to start with invalid configuration.

```
// TODO 1: Create a class called PaymentOptions with two properties:  
//   - GatewayUrl (string, required use [Required] attribute)  
//   - MaxDepositBirr (decimal, range 100-100000 use [Range] attribute)  
// Stuck? public class PaymentOptions { [Required] public required string GatewayUrl { get; init; }  
// ... }
```

```
// TODO 2: In Program.cs, bind PaymentOptions to the "Payments" section of appsettings.json  
// and enable startup validation.  
// Stuck? builder.Services.AddOptions<PaymentOptions>()  
//     .BindConfiguration("Payments")  
//     .ValidateDataAnnotations()  
//     .ValidateOnStart();
```

```
// TODO 3: Test it delete the "Payments" section from appsettings.json and run the app.  
// What error do you see? Does the app start or crash immediately?
```

Run / Call / Expected / Common failure

- **Run:** dotnet run **after** removing or commenting out the entire "Payments" section in appsettings.json (or removing GatewayUrl only, if your validators require it).
- **Call:** (none failure happens at startup)
- **Expected:** Process **does not stay running**; console shows something like:

Microsoft.Extensions.Options.OptionsValidationException: DataAnnotation validation failed for 'PaymentOptions': 'The GatewayUrl field is required.'

- **Common failure app starts anyway:** ValidateOnStart() is missing, or options are not bound to "Payments", or the section name in JSON does not match BindConfiguration("Payments").

Exercise 4: The Unsearchable Logs (Structured Logging)

Context: A student complained that their grade was not saved. The IT team searched Application Insights for logs, but could not filter by the student's ID because the logs were written as raw concatenated strings.

The difference between searchable and unsearchable logs comes down to one habit:

// BAD one concatenated blob, not queryable

```
logger.LogInformation("Enrolling student " + studentId + " in course " + course);
```

// GOOD StudentId and Course become queryable properties in any log aggregator

```
logger.LogInformation("Enrolling student {StudentId} in course {Course}", studentId, course);
```

Your Task: Audit and Fix EnrollmentService Logging

Open EnrollmentService.cs (the file you created at the top of this session). The EnrollAsync method already uses structured logging. Now apply the same discipline across the service and add proper **log levels**.

1. Add a LogWarning to EnrollAsync when the same student enrolls in the same course twice (duplicate check before inserting):

```
public Task<EnrollmentRecord> EnrollAsync(string studentId, string courseCode)
{
    // Check for duplicate enrollment
    var existing = _store.Values
        .FirstOrDefault(e => e.StudentId == studentId && e.CourseCode == courseCode);

    if (existing is not null)
    {
        _logger.LogWarning(
            "Duplicate enrollment attempt {StudentId} already in {CourseCode} (record {EnrollmentId})",
            studentId, courseCode, existing.Id);
        return Task.FromResult(existing);
    }

    var id = Guid.NewGuid().ToString("N")[..8];
    var record = new EnrollmentRecord(id, studentId, courseCode, DateTime.UtcNow);
    _store[id] = record;

    _logger.LogInformation(
```

```

        "Enrolled {StudentId} in {CourseCode} record {EnrollmentId}",
        studentId, courseCode, id);

    return Task.FromResult(record);
}

2. Add a LogWarning to GetByIdAsync when the requested record does not exist:
public Task<EnrollmentRecord?> GetByIdAsync(string id)
{
    _store.TryGetValue(id, out var record);
    if (record is null)
    {
        _logger.LogWarning("Enrollment {EnrollmentId} not found", id);
    }
    return Task.FromResult(record);
}

3. Add a LogInformation to DeleteAsync:
public Task<bool> DeleteAsync(string id)
{
    var removed = _store.Remove(id);
    if (removed)
        _logger.LogInformation("Deleted enrollment {EnrollmentId}", id);
    else
        _logger.LogWarning("Delete failed enrollment {EnrollmentId} not found", id);
    return Task.FromResult(removed);
}

```

Log level decision rules (use these throughout the TMS):

- **Information** a business event completed successfully (enrollment created, deleted)
- **Warning** something unexpected but recoverable happened (duplicate attempt, record not found)
- **Error** an operation failed and needs attention (exceptions, data corruption)

Run / Call / Expected / Common failure

- **Run:** dotnet run, then call POST /api/enrollments twice with the same student and course (you will wire this endpoint in Session 3 / Exercise 5). For now you can verify by calling the methods directly from a temporary test endpoint, or wait until Session 3.
- **Call:** Watch the **console** output.
- **Expected:** First call logs [Information] Enrolled S-001 in CS-101. Second call logs [Warning] Duplicate enrollment attempt. A GET for a nonexistent ID logs [Warning] Enrollment xyz not found.

- **Common failure:** Still seeing concatenated strings you used "Enrolling " + studentId instead of "Enrolling {StudentId}", studentId.
- **Common failure:** All logs show [Information] you forgot to use LogWarning for the edge cases.

Why this session ends here

Session 2 covered three failures that all look the same in production logs: “the app worked yesterday and now it doesn’t.” Every senior engineer has fought each one, often more than once.

- A singleton holding a scoped service is a **lifetime mismatch**. Fix: IServiceScopeFactory and a using scope. Detect early with ValidateScopes = true and ValidateOnBuild = true.
- A missing required configuration value is a **deployment hazard**. Fix: strongly-typed options with [Required] / [Range] and ValidateOnStart(). Detect at process start, never at user-facing failure time.
- A concatenated log string is an **observability hazard**. Fix: named template placeholders {StudentId}, {CourseCode}, with appropriate log levels. Detect by searching for a specific student in the aggregator and seeing zero hits when there should be many.

In Session 3 you build the API surface that makes all of this visible from the outside controllers, the error envelope clients consume, and the environment toggle that decides what diagnostic surface is exposed in Dev versus Production. The capstone refactor at the end of Session 3 will probe whether you can change all three pillars (DI, config, logging) under time pressure that is what makes today’s work matter.

Session 2 Checkpoint

Before moving to Session 3, confirm all four:

- ☐ Registering EnrollmentWorker as singleton with a constructor taking IEnrollmentService directly causes startup to fail with a “Cannot consume scoped service” error (proves ValidateScopes is on)
- ☐ After the IServiceScopeFactory fix, the app starts and 15 concurrent calls to worker-smoke complete with no captive-dependency exceptions
- ☐ Deleting Payments:GatewayUrl from appsettings.json causes the app to crash at startup with a clear OptionsValidationException message not a 500 on first request
- ☐ A second enrollment of the same student into the same course logs [Warning] Duplicate enrollment attempt with {StudentId} and {CourseCode} as queryable structured properties not a concatenated string